



A MAJOR PROJECT REPORT ON

“NoteNest: A Web-Based Note-Taking Application with Admin Panel”

Submitted in partial fulfillment of the requirements for the
Degree of Diploma in Computer Engineering

Sudip Neupane
Umesh Budha
Nabi Akhtar Khan
[16 – Sudip Neupane]
[17 – Umesh Budha]
[10 – Nabi Akhtar Khan]

Department of Computer Engineering
Kathmandu Institute of Technology
Council for Technical Education and Vocational Training (CTEVT)

Shrawan, 2083

“NoteNest: A Web-Based Note-Taking Application with Admin Panel”

A PROJECT REPORT

Submitted by:

Sudip Neupane – 16

Umesh Budha – 17

Nabi Akhtar Khan – 10

Submitted to:

Department of Computer Engineering

Kathmandu Institute of Technology

In partial fulfillment of the requirements for the degree of
Diploma in Computer Engineering

Shrawan, 2083

CERTIFICATE OF APPROVAL

The undersigned certify that they have read, and recommended to the Department of Computer Engineering for acceptance, a project report entitled “NoteNest: A Web-Based Note-Taking Application with Admin Panel” submitted by Sudip Neupane, Umesh Budha, and Nabi Akhtar Khan in partial fulfillment of the requirements for the degree of Diploma in Computer Engineering.

Supervisor

Akash GC, Department of Computer Engineering

External Examiner

Head of Department

Department of Computer Engineering

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our project supervisor, Akash GC, for the continuous guidance, valuable suggestions, and encouragement throughout the development of this project, “NoteNest”.

We are also thankful to the Department of Computer Engineering, Kathmandu Institute of Technology, for providing the resources and environment necessary to complete this project. Finally, we extend our gratitude to our family and friends for their constant support and motivation.

EXECUTIVE SUMMARY

NoteNest is a web-based note-taking application, with an accompanying admin panel, designed to give individual users a simple, reliable place to create and organize their notes while giving administrators the tools to oversee the platform. The system allows administrators to view usage statistics, manage user accounts and roles, moderate all notes, review a role-change audit log, and export data to CSV, while regular users can register, log in, and create, tag, favorite, pin, and delete their own notes.

This report presents the complete development lifecycle of the project, including the problem background, objectives, scope, feasibility study, system requirements, related literature, design methodology, implementation details, testing results, and conclusions drawn from the project. The system was developed using the MERN-style stack (React, Express.js, MongoDB, and Node.js) and follows an Incremental Model of development.

The final system successfully meets its stated objectives, providing a reliable, secure, and user-friendly platform for personal note management with administrative oversight, with recommendations for further enhancements outlined in the concluding chapter.

TABLE OF CONTENTS

1. Project Overview	11
1.1. Introduction.....	11
1.2. Objectives and Scope.....	11
1.2.1. Objectives	11
1.2.2. Scope.....	11
1.3. Project Features.....	11
1.4. Feasibility.....	12
1.4.1. Technical Feasibility	12
1.4.2. Economic Feasibility	12
1.4.3. Operational Feasibility.....	12
1.5. System Requirement	12
1.5.1. Hardware Requirements.....	12
1.5.2. Software Requirements.....	12
2. Literature Review.....	13
2.1. Related Works.....	13
2.2. Research Gap	13
3. Design and Methodology.....	14
3.1. Methodology	14
3.2. System Design	14
3.2.1. System Architecture.....	14
3.2.2. Use Case Diagram.....	14
3.2.3. Database Design.....	15
3.3. Tools and Technologies Used.....	16
3.4. Data Source.....	16
4. Result and Analysis.....	17
4.1. Implementation	17
4.2. Testing.....	17
4.3. Result Analysis	18
5. Conclusion, Recommendation and Limitations.....	19
5.1. Conclusion	19
5.2. Limitations	19
5.3. Recommendations for Future Work.....	19

Appendix A: Sample Source Code	21
Appendix B: Sample Questionnaire / Screenshots	22

LIST OF FIGURES

Figure 3.1: System Architecture Diagram	12
Figure 3.2: Use Case Diagram	14
Figure 3.3: Entity Relationship Diagram	16

LIST OF TABLES

Table 3.1: Functional Requirements	10
Table 4.1: Test Cases and Results	20

LIST OF SYMBOLS

λ – Arrival rate

μ – Service rate

Σ – Summation

ABBREVIATIONS

API – Application Programming Interface

CSS – Cascading Style Sheets

DBMS – Database Management System

ERD – Entity Relationship Diagram

HTML – HyperText Markup Language

SQL – Structured Query Language

UI – User Interface

1. Project Overview

1.1. Introduction

This chapter introduces NoteNest, describing the background and motivation behind the project. Consumer note-taking apps are built for a single user and typically offer no way for an administrator to oversee accounts or content. This project proposes a web-based note-taking application that pairs a familiar personal note-taking experience with a dedicated admin panel for platform oversight.

1.2. Objectives and Scope

1.2.1. Objectives

To design and develop NoteNest, a secure, web-based note-taking application that allows registered users to create, organize, and manage personal notes, while giving administrators a dedicated panel to monitor platform activity and manage users and content.

1.2.2. Scope

The system covers user registration and authentication, note creation/editing/deletion, tagging, favoriting, pinning, and trashing of notes, and an administrator panel for viewing platform statistics, managing users and their roles, managing all notes, tracking role-change history through an audit log, and exporting user/note data to CSV. It does not cover real-time collaborative editing, offline synchronization, or a native mobile application.

1.3. Project Features

Key features include secure JWT-based authentication, role-based access control (user/admin), full note CRUD with tagging, favorites, pinning and trash, and an admin dashboard with growth charts, a role-distribution chart, a live activity feed, searchable/sortable/paginated tables, bulk user actions, inline role editing, a role-change audit log, and CSV export.

Table 3.1: Functional Requirements

ID	Requirement	Priority
FR-1	Users shall be able to register and log in securely (JWT-based authentication).	High
FR-2	Users shall be able to create, edit, and delete their own notes.	High
FR-3	Users shall be able to tag, favorite, pin, and trash notes.	Medium
FR-4	Admins shall be able to view platform statistics and charts.	High
FR-5	Admins shall be able to search, sort, and paginate users and notes.	High

FR-6	Admins shall be able to change a user's role, individually or in bulk.	High
FR-7	The system shall record every role change in an audit log.	Medium
FR-8	Admins shall be able to export users and notes to CSV.	Low
FR-9	The system shall prevent an admin from deleting or demoting their own account.	High

1.4. Feasibility

1.4.1. Technical Feasibility

All frameworks and tools used — React, Node.js, Express.js, and MongoDB — are free, open-source, well-documented, and widely supported, with an active community and abundant learning resources. The development team had prior exposure to JavaScript, which made adopting a JavaScript-based full stack practical within the project timeline.

1.4.2. Economic Feasibility

The technology stack used is entirely free and open-source, so development incurred no licensing cost. MongoDB Atlas and common Node.js hosting platforms offer free tiers suitable for a project of this scale, keeping hosting and maintenance costs minimal relative to the productivity benefits the system provides its users.

1.4.3. Operational Feasibility

The interface follows familiar note-taking app conventions (similar to Google Keep and Notion), so end users require little to no training. The admin panel groups related actions (search, sort, bulk edit) into a single consistent layout, making it easy for a non-technical administrator to operate day-to-day.

1.5. System Requirement

1.5.1. Hardware Requirements

Minimum 4GB RAM, dual-core processor, and a stable internet connection for both development and end-user access.

1.5.2. Software Requirements

Windows/Linux/macOS operating system, a modern web browser (Chrome, Firefox, or Edge), JavaScript with the React and Node.js/Express.js frameworks, MongoDB as the database, and a Node.js-compatible server environment.

2. Literature Review

This chapter reviews existing note-taking systems relevant to this project. It discusses their approach and limitations, identifies the gap this project addresses, and justifies the approach taken in NoteNest.

2.1. Related Works

Google Keep offers a fast, minimal note-taking experience with color-coded notes and reminders, but has no user-management or administrative layer, since it is a single-user consumer product. Evernote provides richer organization through notebooks and tags and supports web clipping, but its free tier is limited and it targets individual productivity rather than group or institutional oversight. Notion offers powerful block-based notes and databases with team workspaces, but its flexibility comes with a steeper learning curve and heavier resource use for a task as simple as personal note-taking. None of these mainstream tools expose an administrator view for monitoring platform-wide usage, managing user roles, or auditing changes — a gap that is common in consumer note-taking products, which are built around a single user rather than a managed multi-user platform.

2.2. Research Gap

Existing note-taking applications are built for individual end users and provide no way for an administrator to oversee accounts, moderate content, or audit sensitive actions such as role changes. NoteNest addresses this gap by pairing a familiar note-taking experience for regular users with a dedicated admin panel that provides usage analytics, user and content management, bulk actions, and an audit trail — features more commonly found in enterprise SaaS products than in typical note-taking apps.

3. Design and Methodology

3.1. Methodology

The project was developed using an Incremental Model. Core functionality — user authentication and basic note CRUD — was built and tested first, after which the admin panel was added as a second increment, followed by further increments for the analytics dashboard, audit log, and CSV export. This approach was chosen because it allowed each working increment to be demonstrated and reviewed by the supervisor before the next set of features was built, reducing the risk of large-scale rework.

3.2. System Design

3.2.1. System Architecture

NoteNest follows a three-tier client-server architecture. The presentation tier is a React single-page application (built with Vite) that runs in the user's browser and communicates with the backend over HTTPS. The application tier is a Node.js server built with Express.js, which exposes a REST API, handles authentication with JSON Web Tokens (JWT), and contains all business logic for notes, users, and the admin panel. The data tier is a MongoDB database, accessed through the Mongoose ODM, which persists users, notes, and audit-log records. Refer to Figure 3.1.

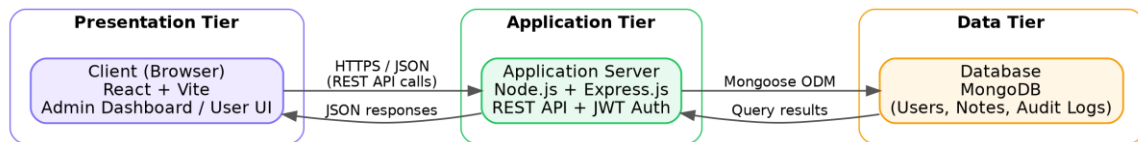


Figure 3.1: System Architecture Diagram

3.2.2. Use Case Diagram

The system has two actors: a regular User and an Admin. A User can register, log in, and create, edit, tag, favorite, pin, and delete their own notes. An Admin has all the privileges of a User, plus access to the admin panel, where they can view dashboard statistics and charts, search/sort/paginate through all users and notes, edit a user's role inline or in bulk, delete users or notes, review the role-change audit log, and export users or notes to CSV. Refer to Figure 3.2.

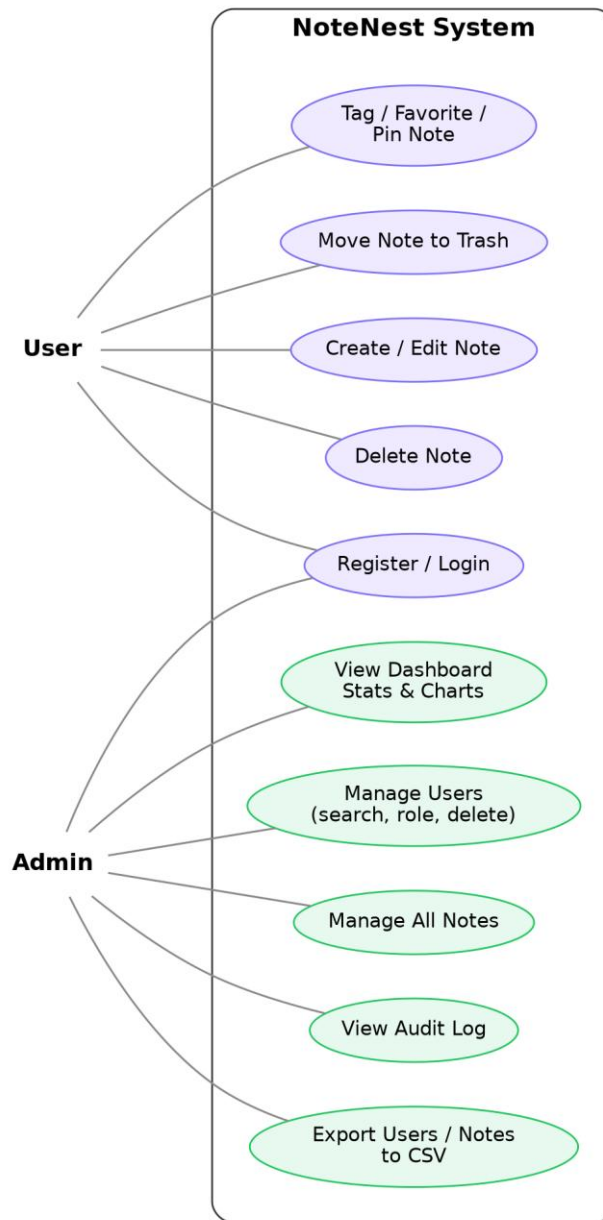


Figure 3.2: Use Case Diagram

3.2.3. Database Design

The database consists of three main collections. The User collection stores each account's username, email, hashed password, and role (user or admin). The Note collection stores each note's title, content, owner reference (user_id), tags, and favorite/pinned/trash flags, with a many-to-one relationship to User. The AuditLog collection records every role change, storing the admin who performed it (performedBy), the user affected (targetUser), the previous and

new role, and a timestamp, with two separate many-to-one relationships back to User. Refer to Figure 3.3.

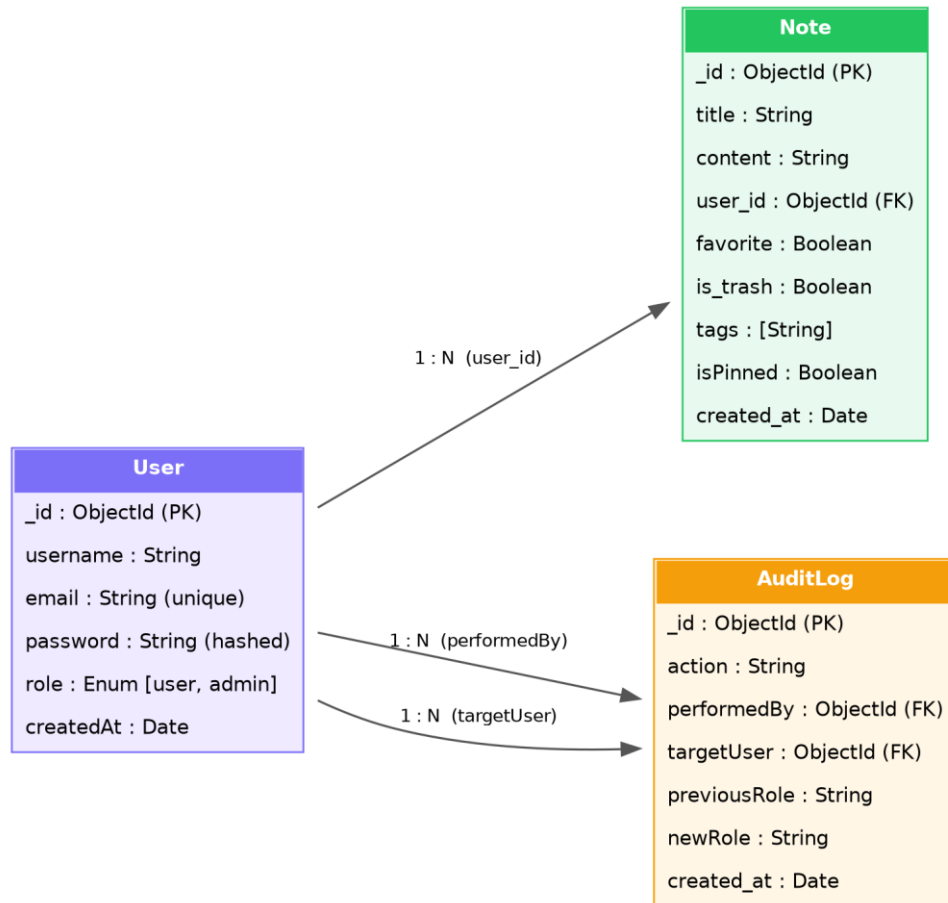


Figure 3.3: Entity Relationship Diagram

3.3. Tools and Technologies Used

Front-end: JavaScript (React, React Router, Vite), Recharts for data visualization, and Lucide React for icons. Back-end: Node.js with Express.js, JSON Web Tokens (jsonwebtoken) and cookie-parser for authentication, and CORS middleware. Database: MongoDB with the Mongoose ODM. Development tools: Visual Studio Code, Git and GitHub for version control, and Postman for API testing.

3.4. Data Source

Test data was generated by the development team through manual use of the application — creating sample user accounts and notes directly through the sign-up and note-creation flows — to validate functionality under realistic, self-produced data during development and testing.

4. Result and Analysis

4.1. Implementation

Implementation proceeded module by module. The authentication module handles registration, login, and JWT-based session verification via middleware that protects private routes. The notes module implements full CRUD for notes, along with tagging, favoriting, pinning, and soft-deletion (trash). The admin module exposes REST endpoints for platform statistics, user and note management, role updates, and audit-log retrieval, all guarded by an admin-only middleware that re-verifies the requester's role against the database on every request. On the front end, the admin panel was implemented as a single dashboard component with tabs for Overview, Users, Notes, and Audit Log, using Recharts for the growth and role-distribution charts and client-side logic for search, sorting, pagination, bulk selection, and CSV export.

4.2. Testing

The system was tested primarily through manual functional testing at three levels: unit-level checks of individual API endpoints using Postman, integration testing of the front end against the live backend and database, and system-level testing of complete user journeys such as registering, logging in, creating notes, and performing admin actions end-to-end. Particular attention was paid to edge cases such as an admin attempting to change or delete their own account, which is explicitly blocked by both the front end and the backend. Refer to Table 4.1 for test cases and results.

Table 4.1: Test Cases and Results

ID	Test Case	Expected Result	Status
TC-1	Register with a new email	Account is created; user is logged in	Pass
TC-2	Login with correct credentials	JWT issued; redirected to dashboard	Pass
TC-3	Login with wrong password	Access denied with an error message	Pass
TC-4	Create a new note	Note appears in the user's note list	Pass
TC-5	Non-admin visits /adminDashboard	Redirected to the regular dashboard	Pass
TC-6	Admin changes another user's role	Role updates; entry appears in audit log	Pass
TC-7	Admin attempts to change own role	Action blocked with an error message	Pass

TC-8	Export filtered users to CSV	Downloaded file matches the filtered rows	Pass
------	------------------------------	---	------

4.3. Result Analysis

Testing confirmed that the system meets the objectives set out in Chapter 1: users can register, authenticate, and manage their own notes, while administrators can view accurate, real-time statistics and manage users and notes through a single interface. The dashboard remained responsive with sample data of a few dozen users and notes, and the interface was found to be intuitive to navigate without prior instruction, since it follows conventions common to other note-taking and admin-panel products.

5. Conclusion, Recommendation and Limitations

5.1. Conclusion

The project successfully delivered a working note-taking application with secure authentication, full note management, and a comprehensive admin panel featuring analytics, user and content management, bulk actions, an audit log, and CSV export — meeting all objectives defined at the start of the project.

5.2. Limitations

The current system has no offline mode and requires an active internet connection at all times. It does not support real-time collaborative editing of a shared note, does not have a native mobile application, and role changes are logged but cannot currently be reverted directly from the audit log.

5.3. Recommendations for Future Work

Future iterations could add real-time collaborative note editing, a native or progressive-web-app mobile experience, rich-text/markdown formatting for notes, note-sharing between users, and the ability to revert a role change directly from the audit log.

REFERENCES

- [1] React documentation. “React.” react.dev (accessed 2026).
- [2] Express.js documentation. “Express – Node.js web application framework.” expressjs.com (accessed 2026).
- [3] MongoDB, Inc. “MongoDB Documentation.” mongodb.com/docs (accessed 2026).
- [4] Mongoose documentation. “Mongoose ODM.” mongoosejs.com (accessed 2026).

APPENDICES

Appendix A: Sample Source Code

Selected key snippets — the admin-only middleware, the role-update controller, and the audit-log schema — are included on the following pages.

A.1 Admin-only middleware (backend/middleware/adminOnly.js)

```
const User = require('../models/User')

const adminOnly = async (req, res, next) => {
  try {
    const userId = req.user?.id || req.user?._id
    if (!userId) {
      return res.status(401).json({ error: 'Access token required. Please log in.' })
    }
    const dbUser = await User.findById(userId)
    if (!dbUser || dbUser.role !== 'admin') {
      return res.status(403).json({ error: 'Admin access required.' })
    }
    req.adminUser = dbUser
    next()
  } catch (err) {
    res.status(500).json({ error: 'Something went wrong verifying admin access.' })
  }
}

module.exports = adminOnly
```

A.2 Role-update controller (backend/controllers/adminController.js)

```
const updateUserRole = async (req, res) => {
  try {
    const { role } = req.body
    if (!['user', 'admin'].includes(role)) {
      return res.status(400).json({ error: "Role must be 'user' or 'admin'." })
    }
    const before = await User.findById(req.params.id)
    if (!before) return res.status(404).json({ error: 'User not found.' })

    if (String(req.adminUser._id) === String(req.params.id)) {
      return res.status(400).json({ error: "You can't change your own role from here." })
    }
    if (before.role === role) {
      return res.json({ id: before._id, role: before.role })
    }

    const updated = await User.findByIdAndUpdate(req.params.id, { role }, { new: true })

    await AuditLog.create({
      action: 'role_change',
      performedBy: req.adminUser._id,
      targetUser: updated._id,
      previousRole: before.role,
      newRole: updated.role,
    })

    res.json({ id: updated._id, role: updated.role })
  }
}
```

```

} catch (err) {
  res.status(500).json({ error: 'Failed to update role.' })
}
}

```

A.3 Audit log schema (backend/models/AuditLog.js)

```

const mongoose = require('mongoose')

const auditLogSchema = new mongoose.Schema({
  action: { type: String, enum: ['role_change'], required: true },
  performedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  targetUser: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  previousRole: { type: String, required: true },
  newRole: { type: String, required: true },
}, {
  timestamps: { createdAt: 'created_at', updatedAt: false },
})

module.exports = mongoose.model('AuditLog', auditLogSchema)

```

Appendix B: Sample Questionnaire / Screenshots

Screenshots of the login page, the user dashboard, and each tab of the admin panel (Overview, Users, Notes, and Audit Log) are included on the following pages.

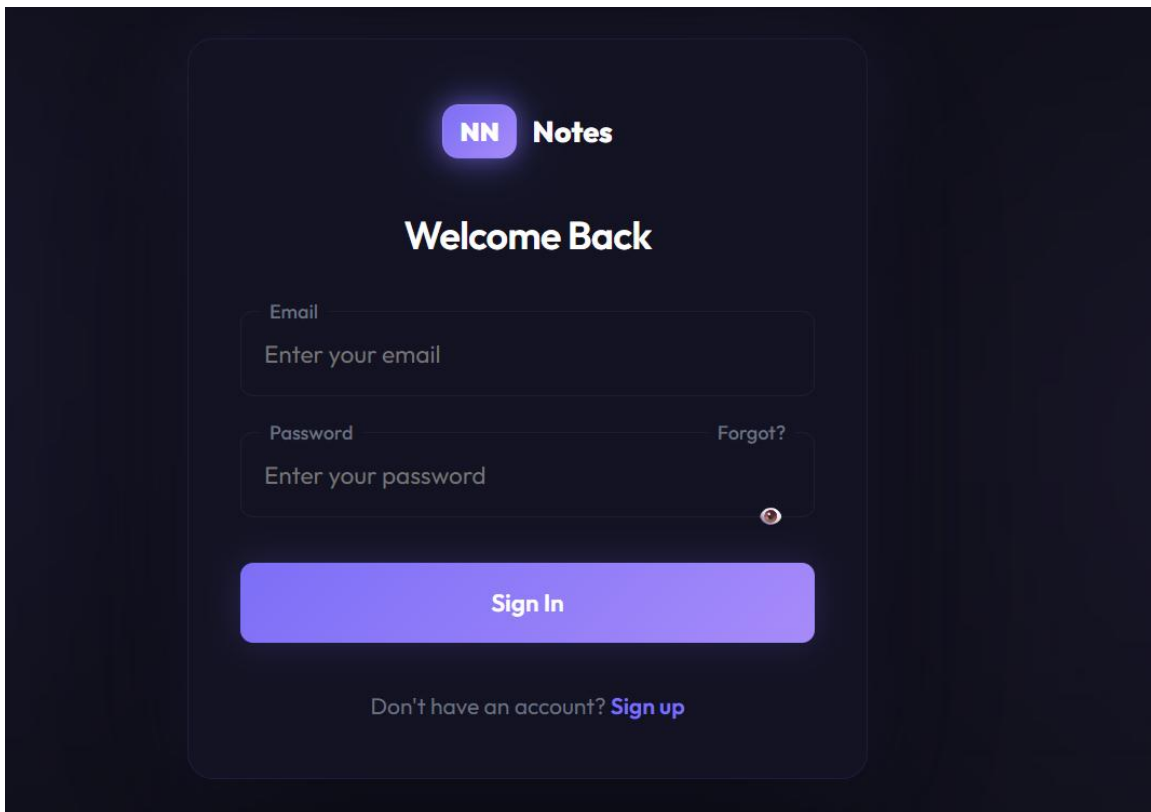
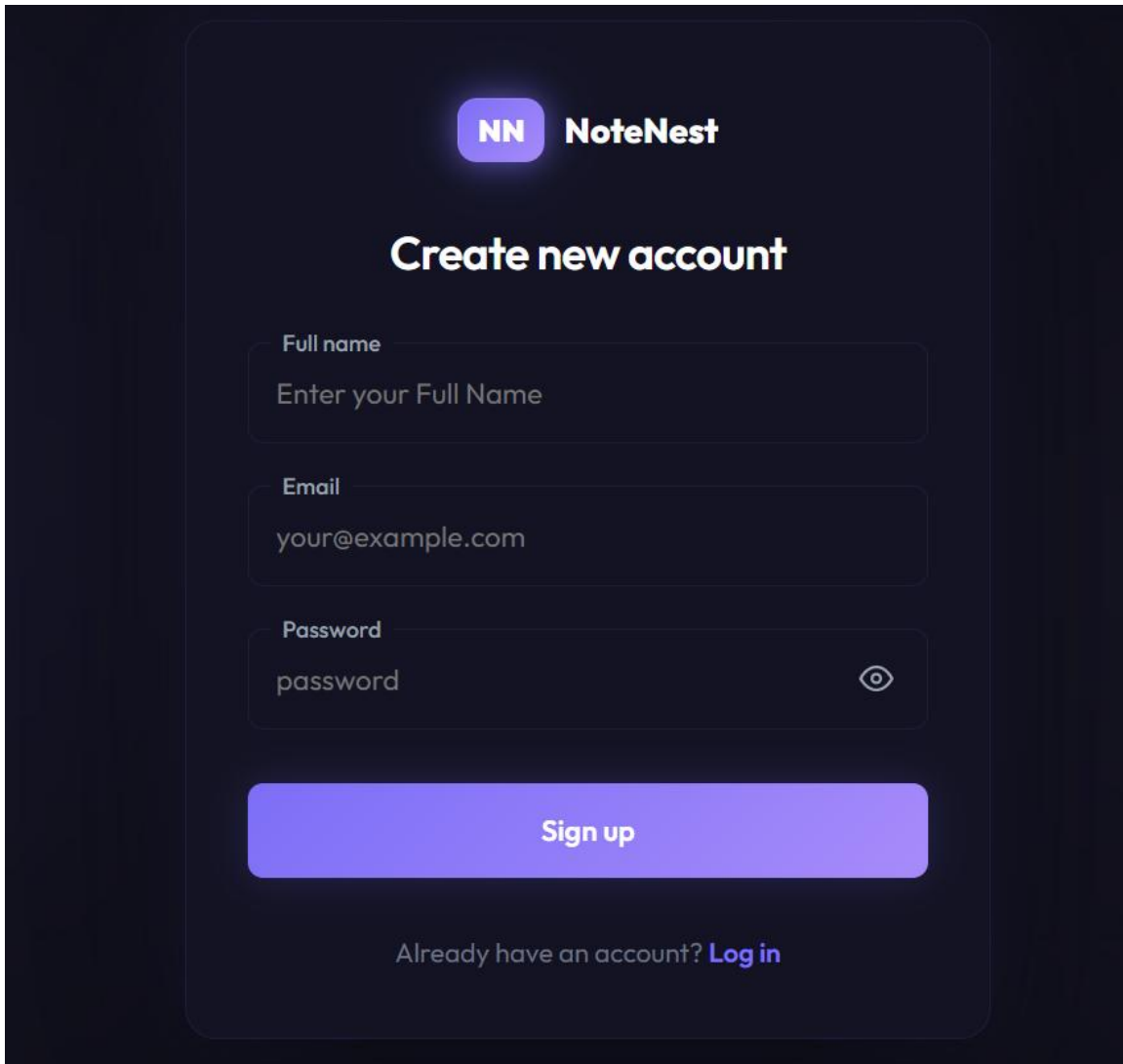


Figure B.1: Login Page



The image shows a dark-themed sign-up page for 'NoteNest'. At the top, there is a logo consisting of a purple rounded square with the letters 'NN' in white, followed by the text 'NoteNest' in a white sans-serif font. Below the logo, the heading 'Create new account' is centered in a large, bold, white font. The form contains three input fields, each with a label above it: 'Full name' with the placeholder 'Enter your Full Name', 'Email' with the placeholder 'your@example.com', and 'Password' with the placeholder 'password'. The password field includes a small eye icon on the right side to toggle visibility. A large, rounded purple button with the text 'Sign up' in white is positioned below the input fields. At the bottom of the form, the text 'Already have an account?' is followed by a blue link that says 'Log in'.

NN NoteNest

Create new account

Full name
Enter your Full Name

Email
your@example.com

Password
password 

Sign up

Already have an account? [Log in](#)

Figure B.2: Sign Up Page

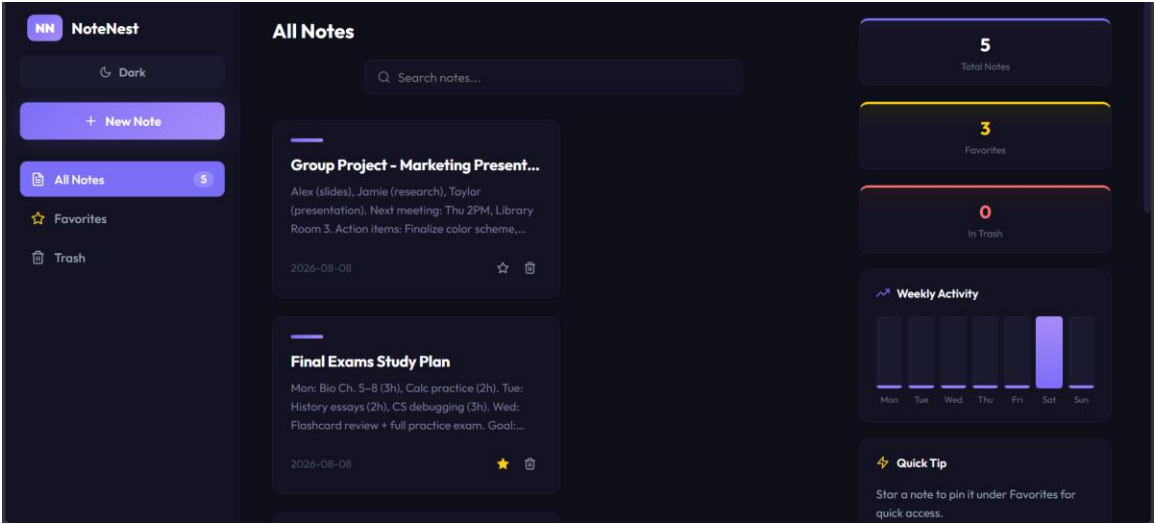


Figure B.3: User Notes Dashboard

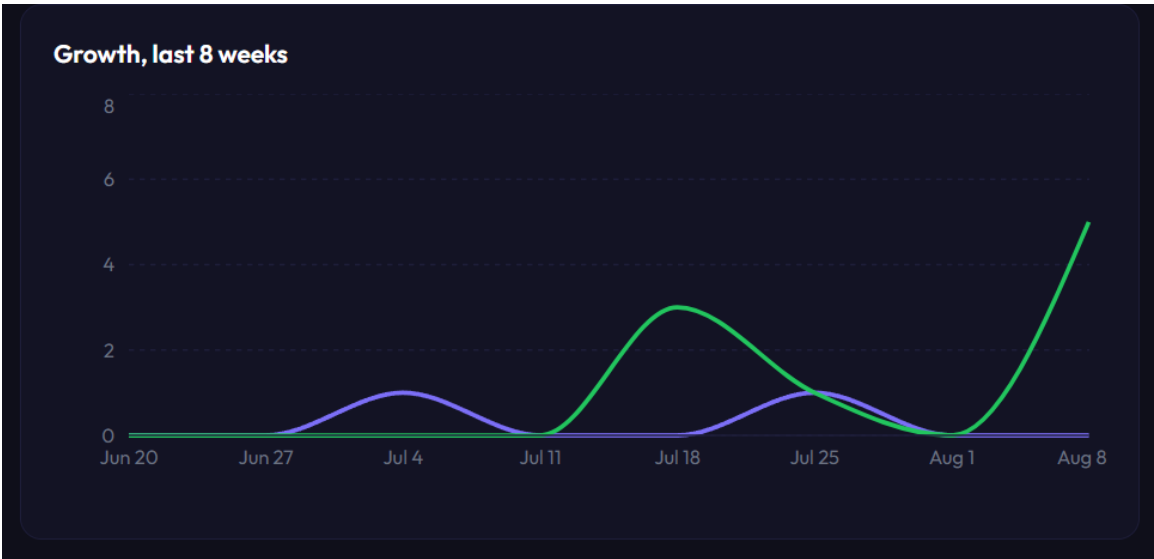


Figure B.4: Admin Overview — Growth Chart

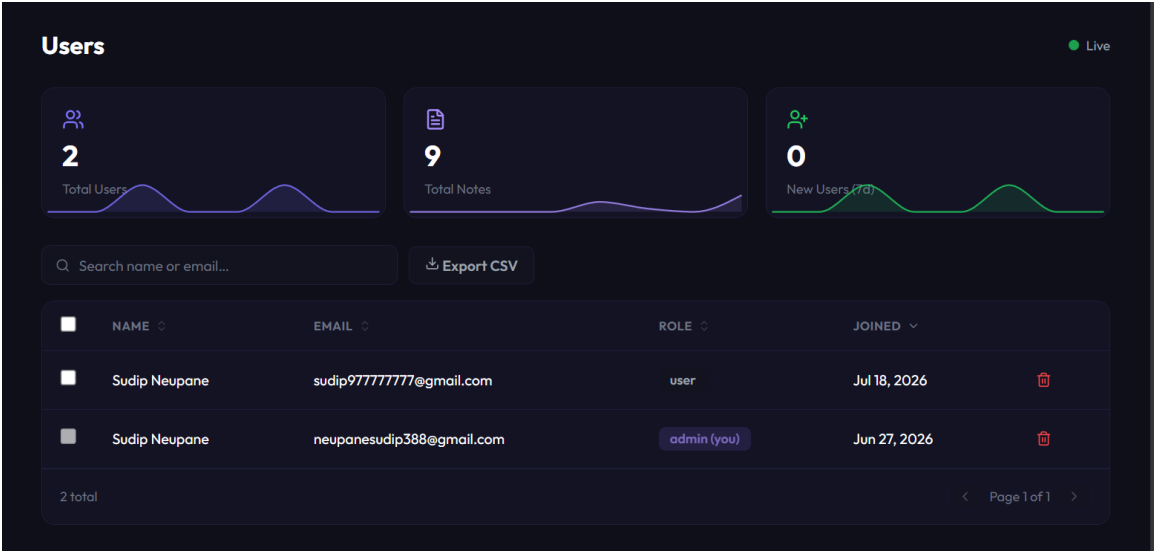


Figure B.5: Admin Panel — Users Tab

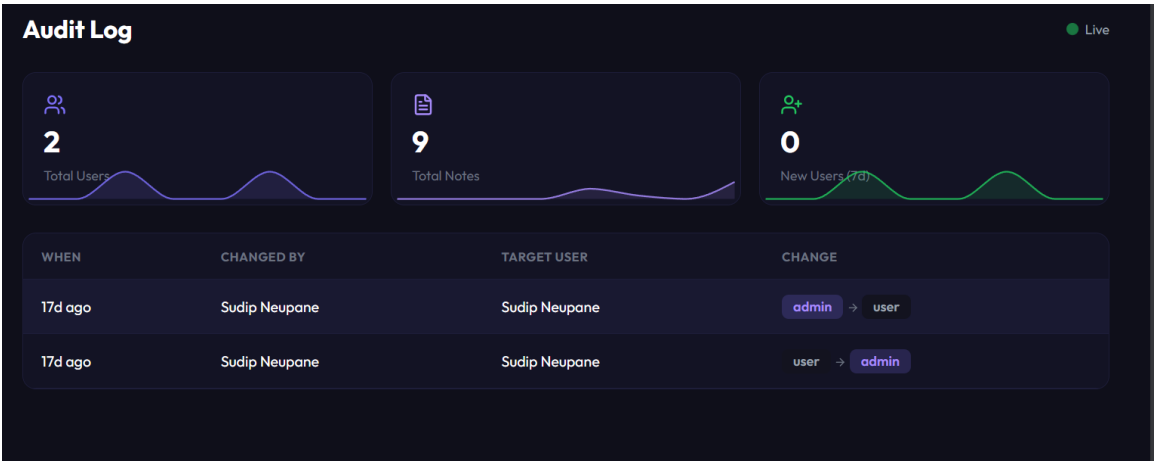


Figure B.6: Admin Panel — Audit Log Tab